

СОДЕРЖАНИЕ

Введение	2
1 Обзор текущего состояния предмета исследования	3
1.1 Обзор зарубежных исследований	3
1.2 Обзор отечественных исследований	5
1.3 Общая характеристика проблемы исследования	9
2 Обзор технологий и методов решения проблемы	11
2.1 Основные методы решения проблемы и их характеристики	11
2.2 Основные технологии решения проблемы и их характеристики	13
2.3 Алгоритм выбора методов и технологий решения проблемы ...	18
3 Формирование требований к решению проблемы	22
3.1 Общая характеристика предлагаемого решения	22
3.2 Функциональные требования к предлагаемому решению	25
3.3 Нефункциональные требования к предлагаемому решению	26
Заключение	28
Список использованных источников	29
Приложение А Блоки кода	30

ВВЕДЕНИЕ

В условиях активного развития информационных технологий и цифровизации образовательного процесса особую актуальность приобретает задача автоматизации проверки программных решений. Традиционные методы оценки задач по программированию требуют значительных временных затрат и подвержены влиянию субъективных факторов.

В рамках данной научно-исследовательской работы рассматривается проблема разработки системы автоматизированной проверки задач, решаемых на языках программирования. Целью работы является анализ подходов к автоматизированной проверке программного кода и формирование требований к программной системе, обеспечивающей объективную, быструю и масштабируемую оценку результатов выполнения задач.

Для достижения поставленной цели в последующих разделах работы будет проведено исследование существующих решений, предметной области, включающее анализ особенностей процесса проверки программных решений и требований, предъявляемых к подобным системам. Также будет выполнен обзор и анализ существующих решений автоматизированной проверки, выявлены их преимущества и недостатки.

Результаты проведённого исследования послужат теоретической и методологической основой для последующего проектирования и реализации системы автоматизированной проверки задач по программированию в рамках выпускной квалификационной работы.

1 Обзор текущего состояния предмета исследования

1.1 Обзор зарубежных исследований

Зарубежные исследования в области автоматизированной проверки задач по программированию в последние годы демонстрируют устойчивый рост интереса к использованию интеллектуальных методов анализа программного кода, что обусловлено увеличением численности обучающихся, усложнением учебных заданий и необходимостью повышения объективности и масштабируемости оценивания. [1] Существенное внимание уделяется сравнению традиционных методов автоматической проверки, основанных на тестировании, с подходами, использующими методы статического анализа и современные языковые модели.

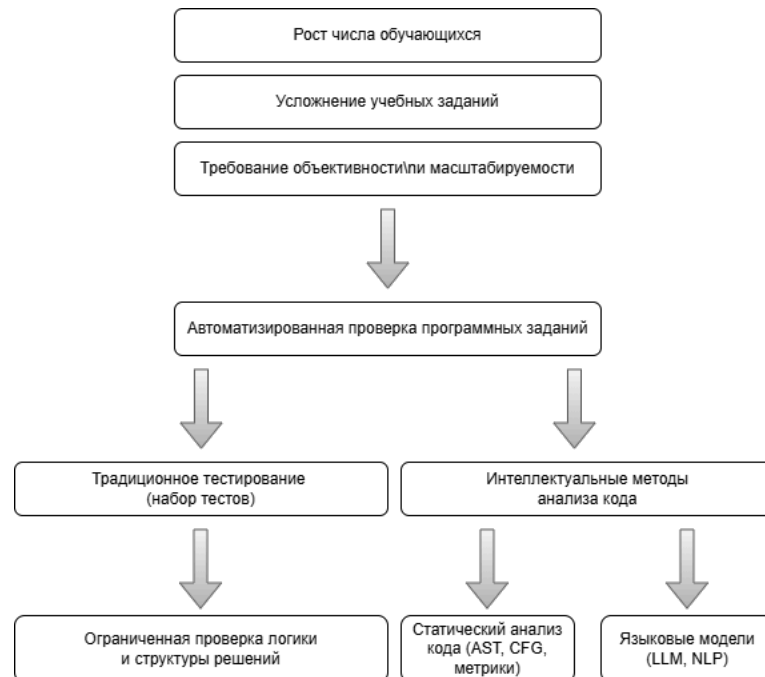


Рисунок 1 — Структурно-логическая схема методов автоматизированной проверки программных заданий

В статье «Automated Grading and Feedback Tools for Programming Education: A Systematic Review» подчёркивается, что классические автоматизированные системы оценки программных заданий в большинстве случаев опираются на динамическое тестирование, при котором проверяется соответствие выходных данных программы ожидаемым результатам для заданного набора входных параметров. [1] Такой подход позволяет эффективно выявлять функциональные ошибки, однако обладает рядом ограничений, связанных с невозможностью оценить частичную корректность решения, качество алгоритма, стиль программирования и соблюдение принятых соглашений.

Ряд исследований указывает на то, что автоматическая тестовая проверка особенно неблагоприятно сказывается на результатах начинающих программистов. Отмечается, что студенты с низким уровнем подготовки часто не получают адекватной обратной связи, поскольку программа не компилируется или не проходит тесты, несмотря на наличие логически верных фрагментов решения. В результате итоговая оценка оказывается ниже той, которую обучаемый мог бы получить при ручной проверке преподавателем.

В качестве альтернативы или дополнения к тестированию в зарубежных работах рассматриваются методы статического анализа программного кода, не требующие его выполнения. Такие подходы основаны на анализе структуры программы, используемых конструкций языка и взаимосвязей между элементами кода. Предлагается сравнивать программные решения на уровне абстрактных представлений, что позволяет выявлять эквивалентные решения, отличающиеся синтаксически, но совпадающие по смыслу.

Другой темой современных исследований является использование больших языковых моделей для автоматизированной оценки программных заданий [2]. В статье «Automatic Evaluation of Programming Tasks Supported by Language Models» рассматривается возможность применения языковых моделей для оценивания фрагментов программного кода на основе заранее заданных критериев и подробных рубрик. В рамках экспериментальных исследований зарубежными авторами было проведено сравнение оценок, полученных с помощью различных языковых моделей, с результатами ручной проверки программных заданий по языкам C++ и Java. Эксперименты включали анализ сотен программных решений, проверенных как автоматическими средствами, так и преподавателями.

Полученные значения коэффициентов корреляции в диапазоне от 0,6 до 0,9 свидетельствуют о том, что языковые модели способны достаточно точно воспроизводить экспертную оценку при корректной формализации критериев оценивания.

В то же время в зарубежных исследованиях подчёркивается, что эффективность применения языковых моделей в значительной степени зависит от качества и детальности оценочных рубрик. Недостаточно формализованные критерии приводят к увеличению числа ложных отрицательных оценок, особенно в случаях, когда преподаватели допускают незначительные синтаксические ошибки или отклонения от строгих требований языка программирования.

В ранее упомянутой зарубежной статье, помимо этого рассматриваются вопросы согласованности оценивания. Отмечается, что ручная проверка, выполняемая несколькими преподавателями, часто сопровождается значительными расхождениями в выставляемых баллах, тогда как использование автоматизированных и LLM-ориентированных систем позволяет повысить воспроизводимость результатов и снизить влияние субъективного фактора.

В целом анализ зарубежных исследований показывает, что на современном этапе наибольший интерес представляет комбинированный подход, сочетающий автоматическое тестирование, статический анализ кода и использование языковых моделей для оценки частичной корректности и качества программных решений. Несмотря на выявленные ограничения, зарубежный опыт подтверждает перспективность применения интеллектуальных методов при разработке систем автоматизированной проверки задач по программированию и формирует основу для дальнейших исследований и практической реализации подобных систем.

1.2 Обзор отечественных исследований

Вопросы автоматизированной проверки решений задач по программированию на протяжении последних лет являются объектом активного исследования в отечественной научной литературе, что обусловлено широким внедрением электронных образовательных платформ, а также необходимостью повышения объективности и масштабируемости процедур контроля знаний обучающихся. В рамках

данных исследований рассматриваются как классические методы тестирования программных решений, так и более сложные подходы, ориентированные на анализ структуры и семантики исходного кода.

В статье «Алгоритм оценки качества решения задач по программированию» основное внимание уделяется проблеме качественного оценивания программных решений, выходящего за рамки простого сопоставления выходных данных с эталонными результатами. [3] Подчёркивается, что традиционное тестирование, основанное на наборе входных данных, позволяет определить корректность решения задачи, однако не даёт возможности оценить уровень сформированности алгоритмического мышления, стиль программирования и степень самостоятельности обучаемого. В связи с этим предлагаются методы, направленные на анализ внутренней структуры программного кода и используемых в нём конструкций.

Значительный вклад в развитие данного направления внесли исследования, посвящённые представлению программного кода в виде формализованных структур. В частности, рассматривается возможность преобразования исходного кода в графовые модели, отражающие связи между операторами, переменными и управляющими конструкциями. Графовые модели позволяют абстрагироваться от синтаксических различий, связанных с именованием идентификаторов или форматированием кода, и акцентировать анализ на самих смысловых связях, формирующих алгоритм решения задачи.

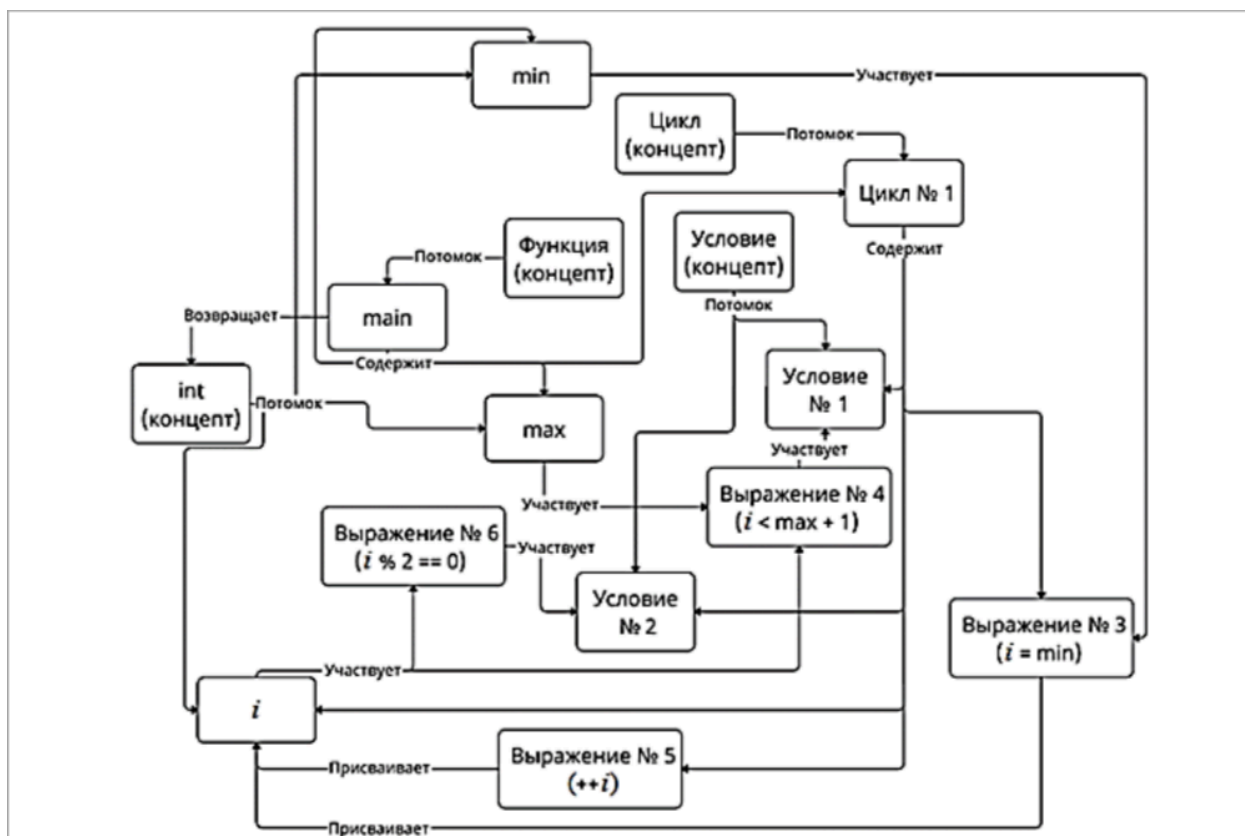


Рисунок 2 — Представление программного кода в виде графовой модели

В работе, посвящённой алгоритму оценки качества решения задач по программированию, предлагается использовать семантические сети как основу для последующего сравнения программных решений. Авторами обосновывается тезис о том, что программный код, несмотря на свою формализованность, может рассматриваться как особый вид языка, элементы которого вступают в устойчивые отношения друг с другом, формируя целостную структуру. На основе данного подхода вводится функция подобия, позволяющая количественно оценить степень схожести двух программных решений, включая эталонное и проверяемое. Дополнительным превосходством подобного метода может заключаться то что он может быть использован не только для оценивания качества решения, но и для выявления заимствований и плагиата.

Отдельное направление отечественных исследований связано с применением методов машинного обучения и, в частности, генеративных нейронных сетей для автоматизированной проверки решений задач по программированию [4]. В статье «Исследование применимости генеративной нейронной сети в качестве классификатора решений заданий по программированию» статье авторами рассматривается возможность использования генеративной нейронной сети в качестве классификатора

корректности программных решений, написанных на языке Python. [A.1] Авторы акцентируют внимание на том, что нейросетевая модель способна анализировать программный код без предварительной подготовки тестов и формальных правил, что принципиально отличает данный подход от классических систем автоматической проверки, которые основываются исключительно на заранее составленных тестах и входных данных.

В рамках указанного исследования нейронная сеть используется для классификации решений по тематическим признакам, таким как использование определённых языковых конструкций и соответствие логики программы условиям задачи. Показано, что для ряда тем базового курса программирования достигаются достаточно высокие значения метрик точности, что позволяет рассматривать данный подход как потенциально применимый в учебной практике. При этом отмечается, что качество классификации существенно зависит от предметной области задачи, формулировки требований и наличия в коде числовых параметров и граничных условий.

Авторы В.М. Мальцев, И.А. Назаров, Е.А. Павлова подчёркивают, что генеративная нейронная сеть в большей степени ориентируется на общую алгоритмическую структуру решения, чем на точное совпадение числовых значений и формальных условий. Это, с одной стороны, позволяет корректно обрабатывать решения, отличающиеся по синтаксису, но реализующие верную логику, а с другой — приводит к снижению точности при анализе задач, требующих строгого соблюдения условий. Таким образом, нейросетевой подход демонстрирует устойчивость к вариативности программных реализаций, но требует аккуратного определения критериев правильности.

В целом результаты данного исследования подтверждают практическую значимость использования интеллектуальных методов анализа программного кода в образовательных системах. Делается вывод о целесообразности применения генеративных нейронных сетей в качестве вспомогательного инструмента автоматизированной проверки, особенно на этапах тренировки навыков программирования и предварительного контроля. Вместе с тем подчёркивается необходимость сочетания нейросетевых методов с формальными алгоритмами анализа кода и заранее заданными критериями оценки, что необходимо для повышения надёжности и достоверности итоговых результатов.

1.3 Общая характеристика проблемы исследования

Процесс обучения программированию и проведения массовых ИТ-соревнований в современных условиях требует обработки огромных массивов данных. Основная сложность заключается в необходимости быстрой и точной проверки программного кода, написанного множеством разных людей. Традиционный метод, при котором преподаватель или эксперт вручную изучает каждое решение, становится физически невозможным при росте числа обучающихся. Ручной аудит неизбежно ведет к задержкам в получении обратной связи, что мешает студентам оперативно исправлять ошибки. Кроме того, человеческий фактор привносит долю субъективности: разные проверяющие могут по-разному оценивать стиль написания кода или оформление алгоритма, что недопустимо для сохранения справедливости в контестах между участниками.

Существующие сегодня автоматизированные системы лишь частично решают эти вопросы, так как большинство из них работает по принципу «черного ящика». Они проводят только функциональное тестирование, сравнивая выходные данные программы с заранее заготовленным эталоном. Такой подход не дает представления о том, как именно был достигнут результат. Учащийся может использовать неэффективные методы, «захардкодить» ответы под конкретные тесты или применить конструкции, которые запрещены условием задачи. В образовательной среде это критично, так как целью является не просто получение верного числа на экране, а освоение конкретных алгоритмических приемов и принципов структурного программирования.

Центральной технической проблемой остается отсутствие инструментов для автоматизированного глубокого синтаксического анализа в популярных системах тестирования. Без использования специализированных парсеров, способных строить абстрактные синтаксические деревья, невозможно программно понять логику решения. Обычный поиск по ключевым словам или регулярные выражения не учитывают контекст и вложенность элементов. Например, простая проверка текста не может достоверно подтвердить, находится ли цикл внутри нужной функции или он закомментирован. Это создает ситуацию, когда организаторы либо вынуждены мириться с поверхностной проверкой,

либо возвращаться к трудоемкому ручному анализу исходных текстов для контроля качества.

Параллельно с анализом структуры кода стоит острый вопрос обеспечения безопасности и стабильности вычислительной инфраструктуры. Запуск произвольного кода, написанного третьими лицами, несет в себе потенциальные угрозы: от случайного повреждения файловой системы до преднамеренных атак на сервер. Без механизмов строгой изоляции выполнение пользовательских программ может привести к неконтролируемому потреблению ресурсов. Обычный бесконечный цикл или ошибка в выделении памяти в коде участника способны парализовать работу всей системы, сделав её недоступной для остальных пользователей.

Дополнительные сложности создает фрагментация программных сред. Отсутствие единого стандарта подготовки окружения приводит к тому, что код, успешно работающий на компьютере студента, не запускается на сервере из-за различий в версиях компиляторов, библиотек или системных вызовов. Это порождает ложные негативные результаты и увеличивает количество апелляций. Таким образом, проблема исследования заключается в необходимости создания комплексного инструмента, который объединит в себе безопасную изолированную среду исполнения, поддержку множества языков и интеллектуальный лексический анализ, позволяющий оценивать не только результат, но и саму архитектуру программного решения.

2 Обзор технологий и методов решения проблемы

2.1 Основные методы решения проблемы и их характеристики

Для создания качественной и всесторонней системы автоматизации проверки программного кода необходимо применять комплексный подход. Он должен сочетать в себе глубокий статический анализ исходного текста с классическим динамическим тестированием в строго контролируемых условиях. Такой симбиоз позволяет не только подтвердить работоспособность программы, но и гарантировать безопасность сервера, а также оценить качество реализации алгоритма. Основными методами в данном контексте выступают динамическое тестирование в изолированной среде и статический анализ на основе лексического разбора.

Метод динамического тестирования является базовым стандартом для большинства проверяющих систем. Он заключается в непосредственном запуске скомпилированного или интерпретируемого кода на заранее подготовленном наборе входных данных. Процесс автоматизирован: система подает на стандартный ввод программы определенные значения и перехватывает результат её работы для сравнения с эталонным ответом. Если выходные данные полностью совпадают, задача помечается как выполненная. Главным достоинством этого метода считается его абсолютная объективность, так как он проверяет фактическую работоспособность алгоритма. Однако существенным минусом является ограниченность оценки: система воспринимает программу как «черный ящик». Она не способна определить, насколько эффективно написан код, соблюдены ли требования к его структуре или не является ли верный ответ результатом случайного совпадения при неверной логике.

Для нейтрализации рисков, связанных с запуском чужого кода, динамическое тестирование всегда реализуется в изолированной среде. Применение технологии контейнеризации, такой как Docker, позволяет создать для каждой проверки герметичное пространство, полностью отделенное от основной операционной системы сервера. Внутри такого контейнера программа пользователя имеет доступ только к тем ресурсам и библиотекам, которые были явно разрешены администратором. Это исключает возможность удаления системных файлов, кражи данных или осуществления сетевых атак. Кроме того, контейнеризация позволяет внедрить систему квот: сервер жестко ограничивает объем доступной

оперативной памяти и выделенное процессорное время. В ситуации, когда код участника содержит бесконечный цикл или вызывает утечку памяти, система мониторинга мгновенно останавливает контейнер, предотвращая деградацию производительности всего сервиса.

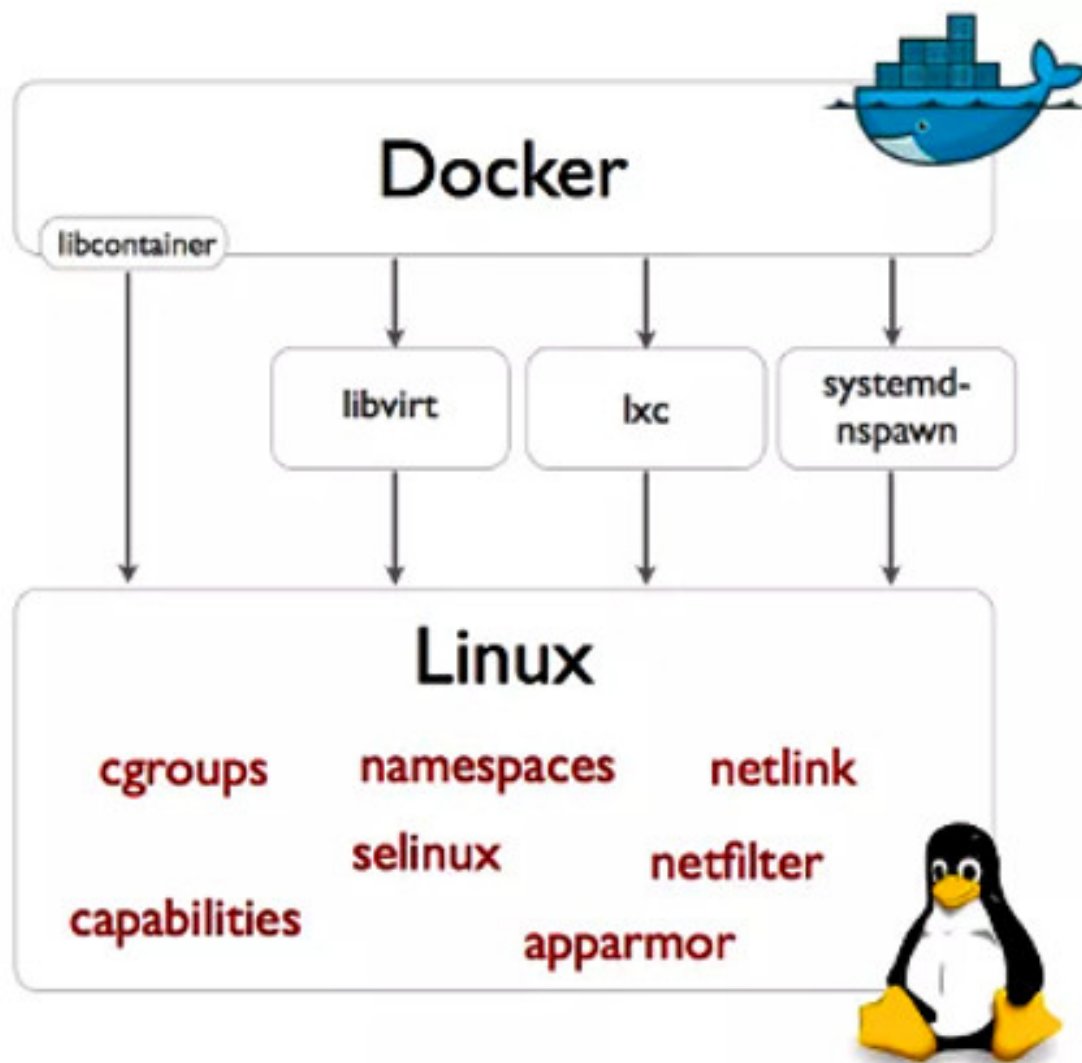


Рисунок 3 — Архитектура среды изоляции Docker и используемые ею низкоуровневые API

Метод статического анализа дополняет проверку, позволяя оценивать программный код без его фактического запуска. В основе данного подхода лежит лексический и синтаксический разбор исходного текста, который может быть реализован с использованием инструмента Tree-sitter. Данный парсер преобразует код в абстрактное синтаксическое дерево, отражающее структуру программы и взаимосвязь её элементов. На основе полученного дерева система может выделять ключевые конструкции языка, такие как функции, переменные, циклы, условные операторы и другие логические элементы. Такой подход позволяет системе более точно

анализировать внутреннее устройство программы и учитывать особенности конкретного языка программирования. Организаторы конкурса получают возможность задавать формализованные требования к структуре решения, например обязательное использование рекурсии или ограничение на применение определённых библиотечных функций. Статический анализ с применением Tree-sitter отличается высокой скоростью обработки и полной безопасностью, так как код анализируется исключительно на уровне структуры и синтаксиса, что исключает выполнение потенциально опасных инструкций и деструктивные действия со стороны пользовательской программы.

Другим перспективным направлением развития системы автоматизированной проверки является внедрение модуля, использующего большие языковые модели (Large Language Models или сокращённо LLM) для оценки программных решений. В отличие от классических методов динамического тестирования и статического анализа, данный подход ориентирован на семантическое понимание кода и его соответствие поставленной задаче. LLM-модуль способен анализировать не только структуру программы, но и логику рассуждений разработчика, выявляя уровень обобщённости решения, корректность выбранного алгоритмического подхода и степень его соответствия условиям задачи. Использование LLM позволит расширить границы оценки качества программного кода за счёт анализа тех характеристик, которые трудно формализовать традиционными методами. Модель может учитывать стиль программирования, осмысленность именования переменных и функций, логичность разбиения кода на блоки, а также потенциальную читаемость и поддерживаемость решения. Другим плюсом является то что LLM может использоваться для выявления типичных ошибок, не приводящих к сбою на тестовых данных, но указывающих на логические неточности или неоптимальные конструкции.

2.2 Основные технологии решения проблемы и их характеристики

Для реализации надежной изоляции программного кода в системе автоматизированной проверки оптимальным выбором является технология Docker. Этот инструмент позволяет запускать решения пользователей в отдельных контейнерах, создавая непреодолимый

барьер между потенциально опасным кодом и основной операционной системой сервера. В такой среде для каждой отдельной проверки динамически формируется абсолютно чистое окружение, содержащее только необходимые компиляторы, интерпретаторы и системные библиотеки. Такой подход помогает полностью избежать конфликтов между разными версиями программного обеспечения и гарантирует, что сервер защищен от случайного или преднамеренного повреждения системных файлов. Docker также предоставляет мощные инструменты для управления ресурсами, позволяя администратору гибко настраивать лимиты на использование оперативной памяти и процессорного времени для каждого запущенного решения. В случае возникновения критических ошибок, таких как бесконечные циклы или утечки памяти, система может мгновенно остановить проблемный контейнер, сохраняя общую работоспособность и стабильность всей платформы.

Использование технологии Docker не только обеспечивает безопасность, но и значительно расширяет функциональные возможности платформы, позволяя поддерживать широкий перечень популярных языков программирования в рамках единого интерфейса. Для каждого языка, будь то Python, Java, C++, Go или Rust, создается специализированный образ контейнера с заранее настроенной конфигурацией, специфичными компиляторами и интерпретаторами. Это предотвращает необходимости установки десятки потенциально конфликтующих зависимостей непосредственно на основной сервер, что упрощает техническое обслуживание всей инфраструктуры.



Рисунок 4 — Опрос среди разработчиков о их выборе в языках программирования

Благодаря такому подходу пользователи получают возможность использовать свой любимый язык программирования для решения задач, выбирая наиболее подходящий и привычный инструмент для достижения результата.

При поиске эффективного способа для детального анализа структуры программного кода интерес представляет библиотека Tree-sitter. Она выполняет роль парсера, который превращает текст программы в конкретное синтаксическое дерево. В отличие от простых инструментов поиска по тексту, данная технология способна распознавать логические связи и иерархию элементов в коде. Это позволяет системе точно идентифицировать функции, циклы или условные переходы вне зависимости от стиля написания программы. Tree-sitter поддерживает множество языков и работает достаточно быстро, чтобы проводить проверку в режиме реального времени. С его помощью можно отслеживать правильность использования конструкций и находить лишние элементы, что делает оценку решения более качественной.

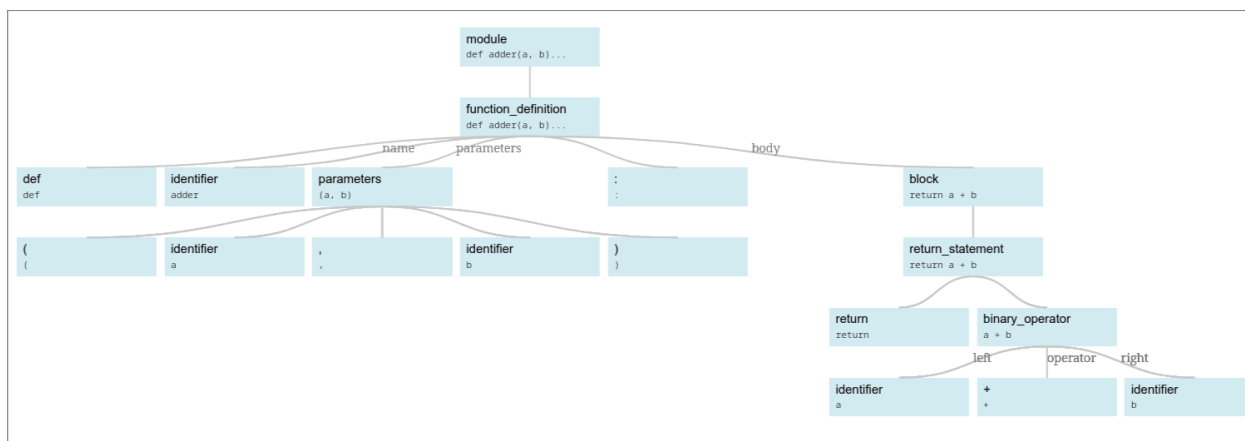


Рисунок 5 — Результат парсинга простой функции при помощи Tree-sitter

Для интеллектуальной оценки качества программного кода и генерации развернутой обратной связи в систему интегрируется локальная предобученная большая языковая модель CodeLlama. В отличие от облачных решений, использование локальной версии модели гарантирует конфиденциальность пользовательских данных и независимость системы от внешних API. CodeLlama специально оптимизирована для задач, связанных с программированием, что позволяет ей анализировать семантическую корректность алгоритмов и предлагать рекомендации по улучшению стиля написания кода. Модель способна оценивать читаемость программы, находить потенциальные логические уязвимости, которые трудно выявить статическим анализом, и объяснять причины ошибок в доступной для пользователя форме. Интеграция подобной LLM-модели позволяет воспроизводить экспертную оценку преподавателя, повышая образовательную ценность платформы за счет глубокой интерпретации каждого решения.

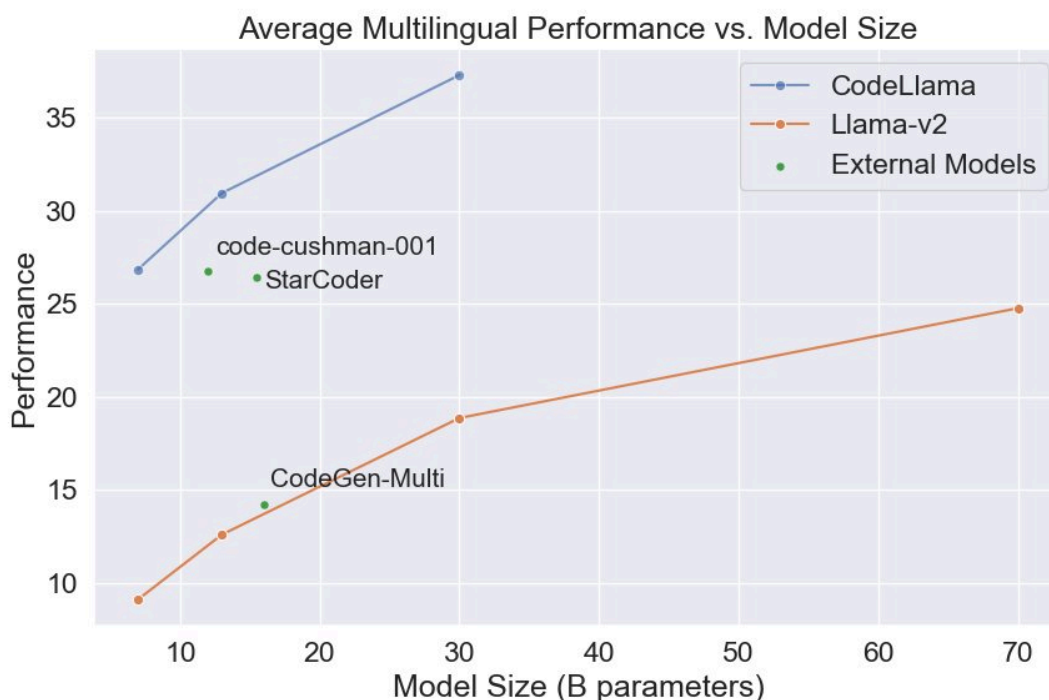


Рисунок 6 — Сравнение эффективности модели CodeLlama с другими

Помимо динамического и семантического анализа, критически важным аспектом проверки является динамическая оценка эффективности алгоритмов через инструментацию кода. Инструментация позволяет внедрять в пользовательское решение специальные счетчики и маркеры, которые фиксируют количество выполненных операций, обращений к памяти или итераций циклов непосредственно в процессе выполнения. В контексте анализа Big O этот метод дает возможность построить зависимость времени выполнения или объема потребляемых ресурсов от размера входных данных. Сравнивая полученные кривые роста с эталонными математическими моделями, система может с высокой точностью определить реальную вычислительную сложность алгоритма, что гораздо надежнее простого замера времени исполнения, подверженного влиянию системных шумов.

Интеграция таких инструментов замера производительности позволяет проводить глубокое сравнение различных реализаций одной и той же задачи. Например, система может автоматически подавать на вход контейнера наборы данных разного масштаба и анализировать, как масштабируется решение пользователя. Если при увеличении входных данных в 10 раз количество операций возрастает в 100 раз, инструментация зафиксирует квадратичную зависимость ($O(n^2)$), что станет основанием для рекомендации по оптимизации кода до линейно-логарифмического

вида. Такой подход превращает автоматизированную проверку из бинарного фильтра «прошел/не прошел» в полноценный инструмент профилирования, помогающий разработчикам осознать разницу между «работающим» и «эффективным» кодом на практических примерах.

Для формирования пользовательского интерфейса отличным выбором станет библиотека React. Она ориентирована на создание отзывчивых веб-приложений, которые обновляются без полной перезагрузки. Это позволяет сделать процесс взаимодействия с системой более плавным, так как результаты тестов и сообщения об ошибках могут появляться в интерфейсе мгновенно. Компонентный подход данной технологии дает возможность удобно расположить на экране все важные элементы: от окна редактора до панели с графиками и результатами анализа.

2.3 Алгоритм выбора методов и технологий решения проблемы

Выбор технологического стека основывается на необходимости совместить высокую производительность веб-интерфейса с безопасным исполнением произвольного программного кода. На первом этапе проводился тщательный анализ требований к изоляции среды. Главной задачей здесь являлся поиск баланса между безопасностью и скоростью работы. Для запуска решений пользователей была выбрана технология контейнеризации Docker. Это решение продиктовано потребностью в жестком ограничении ресурсов и возможности практически мгновенного развертывания чистой операционной системы для каждой новой проверки. В процессе выбора рассматривался вариант использования полноценных виртуальных машин, однако он был отклонен из-за их высокого потребления ресурсов и слишком долгого времени загрузки по сравнению с контейнерами. Docker позволяет обеспечить необходимый уровень защиты ядра основной системы при минимальных накладных расходах на запуск.

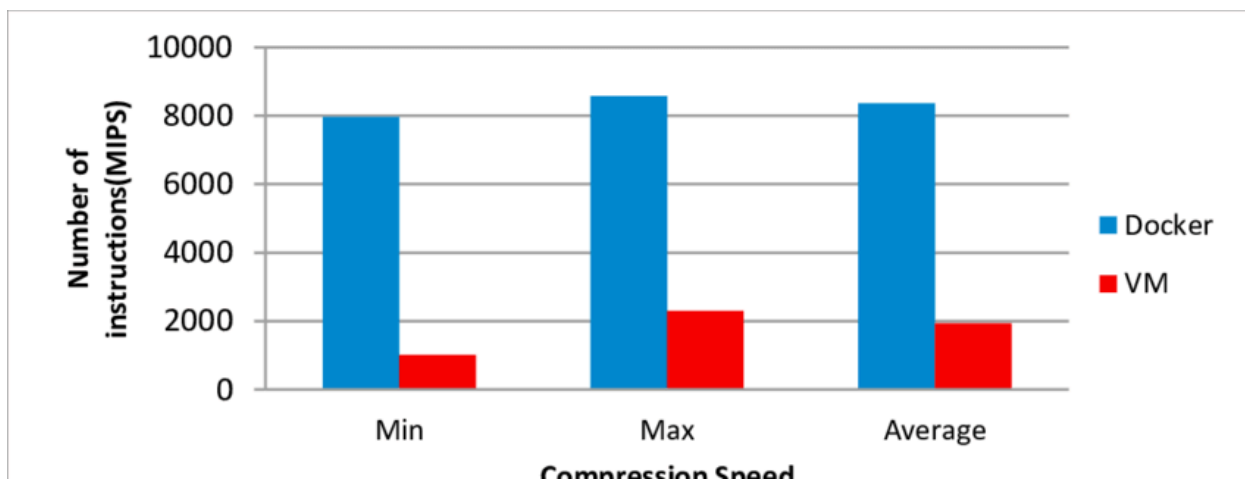


Рисунок 7 – Сравнение скорости исполнения инструкций между Docker и виртуальной машиной

Следующим этапом становится выбор специализированного инструмента для глубокого анализа структуры кода. На этой стадии анализируются методы статического разбора исходных текстов. Вместо написания собственных регулярных выражений, которые часто ошибаются в сложном контексте, или использования простых парсеров, выбор сделан в пользу библиотеки Tree-sitter. Основным критерием здесь стала способность инструмента строить полноценное и точное дерево синтаксиса, которое работает инкрементально и поддерживает широкий спектр современных языков программирования. Этот метод помогает точнее проверять правила написания кода. Система анализирует логику программы: она видит, как функции вложены друг в друга и где действуют переменные. Это надежнее, чем обычный поиск по тексту, потому что алгоритм учитывает структуру файла, а не просто ищет совпадения слов.



[GitHub repository](#)

[Introduction](#)

[Using Parsers](#)

[Creating Parsers](#)

[Syntax Highlighting](#)

[Implementation](#)

[Contributing](#)

[Playground](#)

[Code Navigation Systems](#)

Syntax Tree Playground

Code C++ ☐ Log ☐ Query

```
1 int fib(int n){
2     long long f1 = 1, f2 = 1;
3     int time = (n + 1) / 2;
4     for (int i = 1; i < time; i++){
5         f1 = f1 + f2;
6         f2 = f2 + f1;
7     }
8     if (n % 2 == 0) return f2;
9     else return f1;
10 }
```

Tree 3.7 ms

```
translation_unit [0, 0] - [10, 0]
function_definition [0, 0] - [9, 1]
type: primitive_type [0, 0] - [0, 3]
declarator: function_declarator [0, 4] - [0, 14]
declarator: identifier [0, 4] - [0, 7]
parameters: parameter_list [0, 7] - [0, 14]
parameter_declaration [0, 8] - [0, 13]
type: primitive_type [0, 8] - [0, 11]
declarator: identifier [0, 12] - [0, 13]
body: compound_statement [0, 14] - [9, 1]
declaration [1, 4] - [1, 29]
type: sized_type_specifier [1, 4] - [1, 13]
declarator: init_declarator [1, 14] - [1, 20]
declarator: identifier [1, 14] - [1, 16]
value: number_literal [1, 10] - [1, 20]
```

CSDN @JSLuck

Рисунок 8 — Пример работы лексера Tree-sitter онлайн

Для реализации серверной части и логики управления контекстами выбирается архитектура, способная эффективно справляться с тяжелыми асинхронными задачами. Поскольку компиляция и выполнение кода в контейнере являются длительными операциями, способными замедлить работу сервера, выбор падает на технологии, поддерживающие механизмы очередей сообщений. Это позволяет разделить систему на независимые части: веб-интерфейс, с которым постоянно взаимодействует пользователь, и специализированные вычислительные воркеры, которые занимаются непосредственным выполнением тестов в фоновом режиме. Такой алгоритм разделения зон ответственности исключает зависание сайта в моменты пиковых нагрузок и во время проведения тяжелых вычислений, обеспечивая стабильный отклик платформы для всех участников.

В качестве клиентской части выбирается компонентный подход для создания современного и динамичного интерфейса. Это необходимо для внедрения интерактивных редакторов кода и обеспечения мгновенного отображения статусов проверки без принудительной перезагрузки страниц.

Выбор конкретных библиотек на данном этапе обусловлен их способностью эффективно управлять состоянием приложения и быстро обновлять данные в браузере. Итоговый набор технологий формируется как единая экосистема, где каждый выбранный элемент отвечает за свой конкретный уровень: безопасность исполнения, глубину синтаксического анализа или качество представления данных. Такой алгоритмический подход к выбору стека гарантирует общую надежность, высокую скорость работы и возможность легкого масштабирования платформы в будущем.

3 Формирование требований к решению проблемы

3.1 Общая характеристика предлагаемого решения

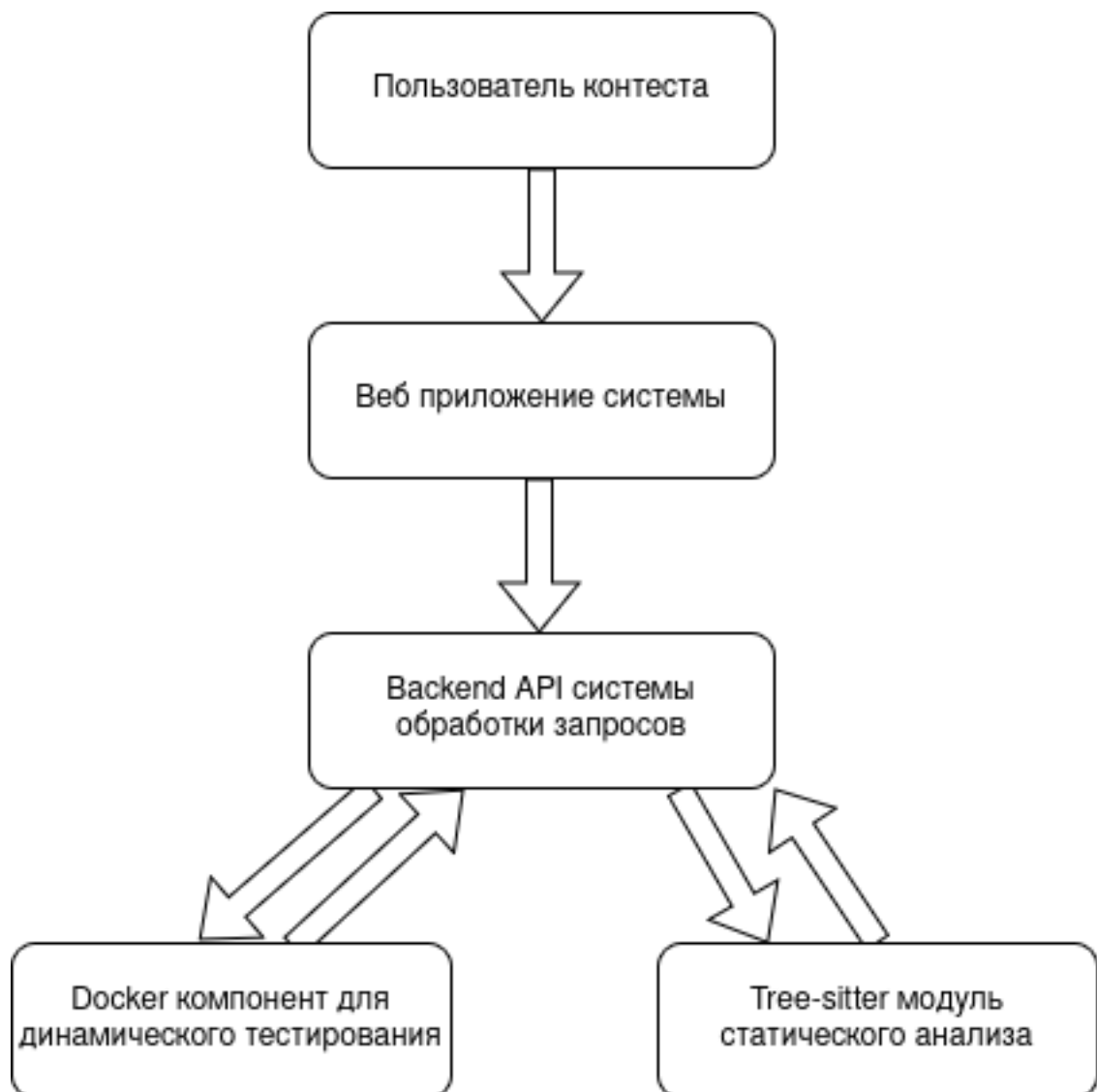


Рисунок 9 – Диаграмма основных компонентов системы

Предлагаемое решение представляет собой современную веб-платформу для автоматизированной проверки программного кода, ориентированную на проведение учебных занятий и соревнований. Основная концепция проекта заключается в создании гибкой цифровой среды, где пользователи могут выступать в роли организаторов или участников конкурсов. Система функционирует на базе клиент-серверной архитектуры, что обеспечивает доступ ко всем инструментам через обычный браузер без установки дополнительного программного обеспечения. После прохождения регистрации в приложении пользователь

получает доступ к личному кабинету, где он может управлять своими мероприятиями или присоединяться к уже созданным испытаниям.

При проектировании нового испытания автор задачи получает в свое распоряжение продвинутый инструментарий для настройки правил проверки. Оценка присланных решений не ограничивается стандартным сравнением текстовых результатов работы программы с эталоном. Организатор имеет возможность задавать сложные критерии, опираясь на глубокий лексический анализ исходного текста. Это открывает возможности для автоматической проверки качества написания кода, контроля использования конкретных алгоритмических конструкций или проверки соблюдения строгих правил синтаксиса, установленных условием задачи.

Сам процесс проверки инициируется автоматически в момент отправки участником своего кода на сервер. Для обеспечения максимальной безопасности и стабильности инфраструктуры каждое присланное решение запускается внутри полностью изолированного Docker-контейнера. Такое архитектурное решение полностью исключает риск повреждения системных файлов сервера вредоносным кодом и позволяет администраторам жестко контролировать потребление ресурсов процессора и памяти. Внутри контейнера создается чистое окружение, где программа компилируется и выполняется на наборе тестовых данных, подготовленных автором задачи.

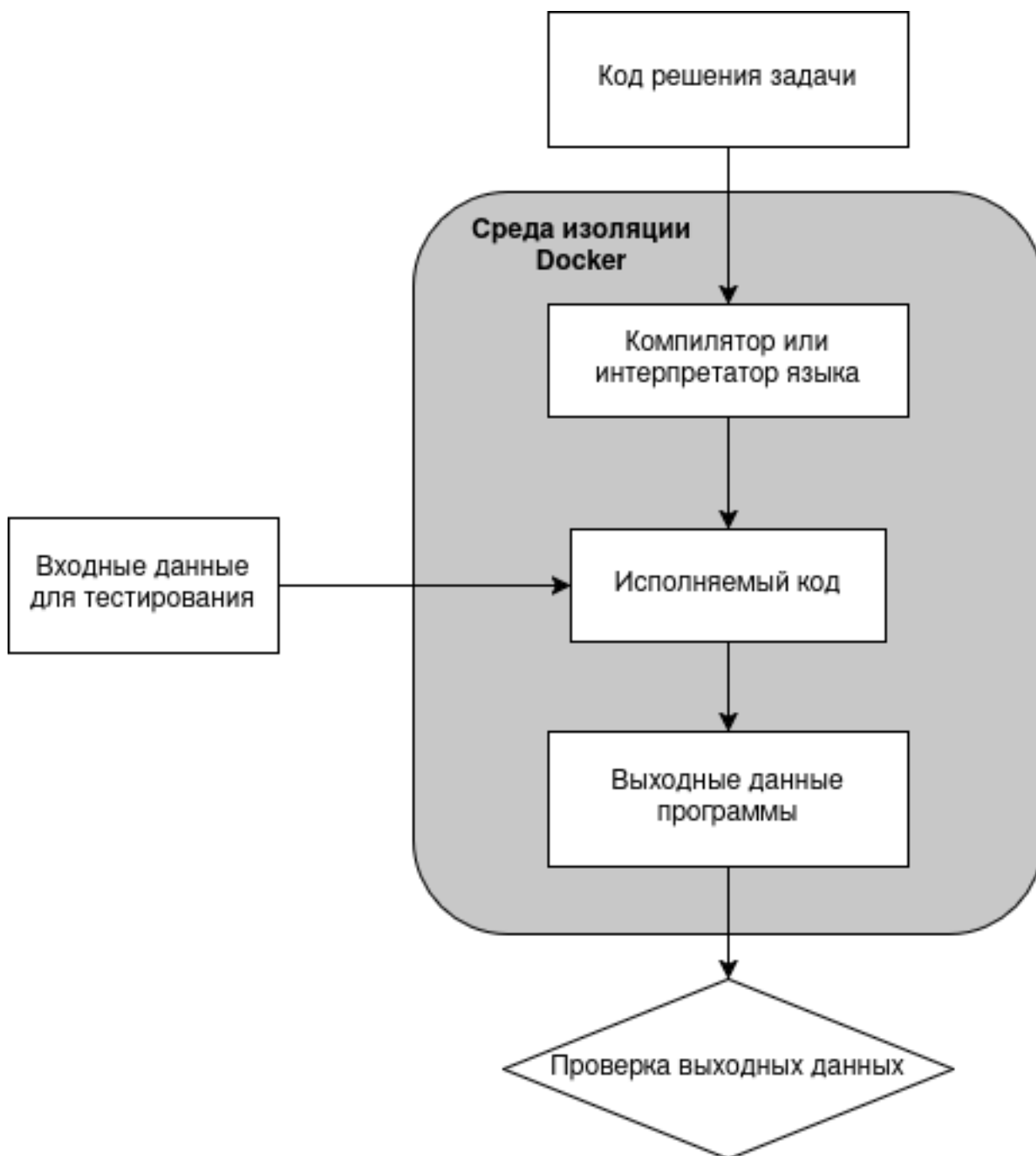


Рисунок 10 — Изоляция программного кода решения

Параллельно с динамическим выполнением программы система запускает модуль статического анализа структуры исходного кода. Для этих целей интегрируется библиотека Tree-sitter, которая мгновенно выстраивает абстрактное синтаксическое дерево (AST) присланного решения. Этот инструмент позволяет проводить детальный разбор кода на уровне логических связей, выявляя наличие или отсутствие необходимых элементов, таких как циклы, рекурсия или специфические типы данных. Итоговый вердикт системы формируется автоматически на основе совокупности результатов: успешности прохождения функциональных

тестов и соответствия кода заданным лексическим критериям. Вся процедура занимает считанные секунды, благодаря чему участник получает мгновенную и подробную обратную связь о качестве своего программного решения.

3.2 Функциональные требования к предлагаемому решению

Система должна обеспечивать полноценный цикл управления учебными и соревновательными мероприятиями по программированию. Для этого в приложении реализуется строгое разграничение прав доступа и специфический набор инструментов для различных категорий пользователей. Архитектура системы обязана поддерживать бесперебойную работу всех модулей — от личных кабинетов до узлов автоматической проверки.

Программный продукт обязан поддерживать надежный механизм регистрации и аутентификации пользователей. После создания учетной записи каждый участник получает доступ к личному профилю, который служит центральным узлом для хранения истории его достижений, статистики решенных задач и созданных мероприятий. Система должна четко разделять права доступа: обычные участники могут только просматривать доступные контесты и отправлять решения, в то время как организаторы получают расширенные права на управление контентом, редактирование условий задач и просмотр детальной аналитики по всем участникам.

Создатель мероприятия должен иметь в своем распоряжении удобный конструктор для гибкой настройки параметров каждой задачи. Программа должна позволять загружать наборы тестовых входных и выходных данных, которые станут основой для классического функционального тестирования. Особое внимание уделяется интерфейсу настройки правил лексического анализа. Организатору необходим визуальный инструмент для выбора конкретных конструкций языка программирования, наличие или отсутствие которых будет напрямую влиять на итоговый балл. Это включает в себя возможность задавать обязательное использование определенных циклов, функций или структур данных, а также накладывать ограничения на использование запрещенных методов.

Техническая часть системы обязана гарантировать абсолютно безопасное и изолированное исполнение присланного кода. Для

реализации этого требования система должна автоматически инициировать запуск новой изолированной среды для каждой попытки сдачи задания. Программная логика должна обеспечивать перехват любых системных сообщений, включая ошибки компиляции и исключения времени выполнения, с последующей их передачей пользователю в структурированном и понятном виде. Встроенный анализатор на базе библиотеки Tree-sitter должен работать в автоматическом режиме, мгновенно выстраивая дерево синтаксиса и проводя его сопоставление с критериями, которые были заданы автором задачи на этапе её создания.

Для пользователей, выступающих в роли решателей, необходимо реализовать интуитивно понятную форму отправки исходного кода. Система должна поддерживать возможность выбора языка программирования из широкого списка доступных вариантов, адаптируя под него правила подсветки синтаксиса в редакторе. Сразу после нажатия кнопки отправки решения участник должен иметь возможность наблюдать за статусом проверки в режиме реального времени через интерактивную панель уведомлений. Полная история всех совершенных попыток с подробным описанием допущенных ошибок, пройденных тестов и начисленных баллов за лексическое соответствие должна быть постоянно доступна пользователю в интерфейсе текущего конкурса.

3.3 Нефункциональные требования к предлагаемому решению

Система должна обладать высокой степенью надежности и исключительной стабильностью при работе в условиях пиковых нагрузок. Это требование критически важно для проведения массовых соревнований и олимпиад, когда сотни участников одновременно отправляют программный код на проверку. Программная архитектура обязана использовать отказоустойчивый механизм очередей сообщений, который гарантирует, что каждая присланная задача будет принята и обработана строго по порядку. Даже при временном выходе из строя отдельных вычислительных узлов данные о попытках не должны быть потеряны, а процесс тестирования должен возобновляться автоматически после восстановления связи.

Безопасность является фундаментальным требованием для платформы такого типа. Изоляция исполняемого кода внутри Docker-контейнеров должна обеспечивать полную герметичность среды.

Необходимо исключить любую возможность несанкционированного доступа пользовательской программы к конфигурационным файлам сервера, сетевым сокетам или процессам других участников. Программный комплекс обязан принудительно ограничивать время выполнения кода (Time Limit) и объем доступной оперативной памяти (Memory Limit) для каждого теста. Эти меры предосторожности защитят серверную инфраструктуру от критических сбоев, вызванных ошибками в коде пользователей, такими как бесконечные циклы, неконтролируемая рекурсия или попытки преднамеренного переполнения памяти.

Важным аспектом выступает масштабируемость и общая производительность системы. Архитектура приложения должна быть спроектирована таким образом, чтобы при росте количества пользователей можно было легко добавлять новые вычислительные воркеры без остановки сервиса. Процесс лексического анализа через Tree-sitter должен быть оптимизирован так, чтобы время построения синтаксического дерева не оказывало заметного влияния на общую задержку проверки. Время отклика интерфейса при навигации по разделам сайта и просмотре условий задач должно оставаться минимальным, обеспечивая комфортную работу пользователей даже при низкой скорости интернет-соединения.

Удобство использования и кроссплатформенная доступность также входят в число приоритетных требований. Интерфейс платформы должен быть интуитивно понятным, чтобы новые участники могли приступить к решению задач без долгого изучения документации. Адаптивная верстка обязана гарантировать корректное отображение всех элементов управления, включая редактор кода, на различных устройствах — от мониторов ПК до планшетов. Код веб-приложения необходимо оптимизировать для стабильной работы во всех популярных современных браузерах. Кроме того, система должна обладать высокой степенью расширяемости, позволяя разработчикам внедрять поддержку новых языков программирования и дополнительных критериев анализа через модульные дополнения, не затрагивая при этом основное ядро платформы.

ЗАКЛЮЧЕНИЕ

В ходе проведённого исследования было изучено как сегодня устроены системы автоматической проверки кода. Мы изучили опыт российских и зарубежных разработчиков и поняли, что обычных тестов, заготовленных создателем конкурса, уже не хватает. Они проверяют только правильный ответ, но не видят саму структуру программы или стиль написания. Из-за этого студент может получить низкий балл за работающий код, в котором просто не соблюдены формальные правила именования. К тому же запуск чужих программ на сервере всегда связан с риском взлома, а инструментов для глубокого и безопасного анализа текста пока мало.

Для создания надёжной системы мы выбрали связку Docker и Tree-sitter. Docker создаёт изолированную «песочницу», где код участника запускается безопасно и не может навредить серверу. Это решение позволяет легко добавлять поддержку разных языков программирования. Библиотека Tree-sitter помогает разбирать код на уровне синтаксиса. Она строит дерево программы, поэтому система начинает учитывать логику вложенности функций и области видимости переменных, а не просто ищет совпадения слов. Чтобы пользователям было удобно работать с системой в реальном времени, интерфейс решили делать на библиотеке React.

На основе этих технологий мы составили список функциональных требований. Главная задача была в том, чтобы сделать проверку гибкой. Организаторы соревнований или преподаватели смогут задавать сложные критерии качества, а участники будут получать подробный отчет об ошибках сразу после отправки решения. Это делает автоматическую оценку более человечной и полезной для обучения, так как студент видит не просто вердикт «ошибка», а понимает, в каком месте кода он допустил просчет.

Эти наработки станут фундаментом для сборки полноценного веб-приложения. Результаты данной работы могут быть использованы для создания современной системы оценки решений задач, написанных на языках программирования. Такая система может быть интегрирована в вузы для проверки заданий студентов, в школы для оценки заданий учащихся и в олимпиады для составления рейтинга лучших решений по множеству критериев.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Messer M. и др. Automated Grading and Feedback Tools for Programming Education: A Systematic Review // ACM Trans. Comput. Educ. New York, NY, USA: Association for Computing Machinery, 2024. Т. 24, № 1.
2. Karsa Z., Goldschmidt B. Automatic Evaluation of Programming Tasks Supported by Language Models // IEEE Access. 2025. С. 1.
3. Александр Сергеевич Федоров Е.Ю.А. Алгоритм оценки качества решения задач по программированию [Электронный ресурс]. URL: <https://cyberleninka.ru/article/n/algoritm-otsenki-kachestva-resheniya-zadach-po-programmirovaniyu>.
4. В. М. МАЛЬЦЕВ Е.А.П. И. А. НАЗАРОВ. ИССЛЕДОВАНИЕ ПРИМЕНИМОСТИ ГЕНЕРАТИВНОЙ НЕЙРОННОЙ СЕТИ В КАЧЕСТВЕ КЛАССИФИКАТОРА РЕШЕНИЙ ЗАДАНИЙ ПО ПРОГРАММИРОВАНИЮ [Электронный ресурс]. URL: https://elib.utmn.ru/jspui/bitstream/ru-tsu/34796/1/miim_2024_92_98.pdf.
5. Latif A. и др. Comparison of Leading Language Parsers – ANTLR, JavaCC, SableCC, Tree-sitter, Yacc, Bison. 2023. Сс. 7–13.
6. Опрос среди разработчиков составленный интернет порталом StackOverflow [Электронный ресурс]. 2025. URL: <https://survey.stackoverflow.co/2025/technology>.

ПРИЛОЖЕНИЕ А

Блоки кода

Ты автоматизированная система для проверки кода студентов на языке Python. Твоя работа – проверять код на решение поставленных целей в задаче. Тебе будут присылать сообщения содержащие описание задачи и код который необходимо будет проверить. Отвечай "Верно" или "Неверно".

Присланные задачи будут на тему "Циклы While".

Проверяй, чтобы в коде решения обязательно было использовано ключевое слово `while`.

Если используется другой цикл для решения задачи, задача считается решенной "Неверно".

Если в описании задачи не предполагается использования бесконечных циклов с условием прерывания, то считай задачи с бесконечным циклом решенными "Неверно".

Листинг А.1 — Пример запроса к генеративной нейронной сети для оценки программного кода